

Cadenas

Programación I

UNRN

Universidad Nacional
de Río Negro

15

Setiembre

e

Programación I

Clase 11: Matrices y memoria.

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Planificación y objetivos a evaluar

01/09/2026	Arreglos de tipos primitivos	Declaración de vectores. Recorridos. Pasar vectores como parámetro. Prototipos de función. Archivos con headers.	3/9/2026	Arreglos de N dimensiones	Matrices. Declaración, recorridos y problemas. Actualización E/S
08/09/2026	Memoria dinámica	Estructura de la memoria ampliada. Heap. malloc, calloc, realloc y free. sizeof y cast. Punteros void, operaciones, aritmética de punteros.	10/9/2026	Arreglos dinámicos I y II	Declaración dinámica de arreglos y matrices. Problemas dimensionados en tiempo de ejecución. instalar compilador, ejercicios
15/09/2026	Estructuras.	Declaración y uso de estructuras de datos. Acceso a los componentes. Operaciones, creación dinámica. sizeof. Uniones.	17/9/2026	Estructuras II	Diccionarios. Registros. Escritura en archivos (binarios y de texto). FIN
22/09/2026	Recup I	recuperatorio 1er parcial.	24/9/2026	Repaso	Repaso: problemas con cadenas, vectores y matrices dinámicos, estructura

- Crear, acceder, operar, imprimir, vectores y matrices estáticos.
 - float matrix[N][M];
- Crear, acceder, operar, imprimir, vectores y matrices dinámicos.
 - float * matrix = malloc(N*M*sizeof(float))
 - ¿qué otra forma de crear una matriz?
- Pensar un algoritmo para resolver un problema.

Aplicando todo lo visto antes:

- Documentación (params, pre y post).
- Modularizar en funciones.
- Argumentos.

Planificación y objetivos a evaluar

01/09/2026	Arreglos de tipos primitivos	Declaración de vectores. Recorridos. Pasar vectores como parámetro. Prototipos de función. Archivos con headers.	3/9/2026	Arreglos de N dimensiones	Matrices. Declaración, recorridos y problemas. Actualización E/S
08/09/2026	Memoria dinámica	Estructura de la memoria ampliada. Heap. malloc, calloc, realloc y free. sizeof y cast. Punteros void, operaciones, aritmética de punteros.	10/9/2026	Arreglos dinámicos I y II	Declaración dinámica de arreglos y matrices. Problemas dimensionados en tiempo de ejecución. instalar compilador, ejercicios
15/09/2026	Estructuras.	Declaración y uso de estructuras de datos. Acceso a los componentes. Operaciones, creación dinámica. sizeof. Uniones.	17/9/2026	Estructuras II	Diccionarios. Registros. Escritura en archivos (binarios y de texto). FIN
22/09/2026	Recup I	recuperatorio 1er parcial.	24/9/2026	Repaso	Repaso: problemas con cadenas, vectores y matrices dinámicos, estructura

- Crear, acceder, operar, imprimir, vectores y matrices estáticos.
 - `float matrix[N][M];`
- Crear, acceder, operar, imprimir, vectores y matrices dinámicos.
 - `float * matrix = malloc( N*M*sizeof(float))`
 - ¿qué otra forma de crear una matriz?
- Pensar un algoritmo para resolver un problema.
- Operaciones con cadenas de caracteres. Cadenas seguras.

Aplicando todo lo visto antes:

- Documentación (params, pre y post).
- Modularizar en funciones.
- Argumentos.

Memoria dinámica: de resumen

Función	Parámetros	Estado Inicial	Uso Principal
<code>malloc()</code>	<code>size_t size</code>	Contenido basura	Reserva rápida de bloques continuos de memoria.
<code>calloc()</code>	<code>size_t num, size_t size</code>	Inicializado en 0	Arreglos donde se requiere garantía de valor inicial cero.
<code>realloc()</code>	<code>void* ptr, size_t size</code>	Conserva datos previos	Agrandar o achicar bloques dinámicos existentes.
<code>free()</code>	<code>void* ptr</code>	Libera el bloque	Devolver memoria al Heap y prevenir fugas (<i>memory leaks</i>).

Memoria dinámica: resumen

Ciclo de Vida Dinámico

Este programa demuestra el flujo completo y profesional recomendados para la gestión de memoria:

1. **Asignación inicial:** Reserva espacio para 3 enteros en cero usando `calloc`.
2. **Redimensionado seguro:** Expande el arreglo a 6 elementos mediante puntero temporal `tmp`.
3. **Liberación limpia:** Libera la memoria con `free` y asigna `NULL` para evitar *dangling pointers*.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    size_t n = 3;
    int *arr = calloc(n, sizeof(int));
    if (arr == NULL) return EXIT_FAILURE;

    // Uso y expansión segura de memoria
    int *tmp = realloc(arr, 6 * sizeof(int));
    if (tmp != NULL) {
        arr = tmp;
    } else {
        free(arr);
        return EXIT_FAILURE;
    }

    // Liberación y neutralización
    free(arr);
    arr = NULL;
    return EXIT_SUCCESS;
}
```

Ejercicio integrador: transponer matrices.

Matriz transpuesta:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}$$

$$\text{dest}[i][j] = \text{orig}[j][i]$$

Ejercicio integrador: transponer matrices.

1. Resolver los ejercicios.
2. Comprimir y enviar por mail con nombre y apellido de los integrantes.

Ejercicio integrador: transponer matrices.

Requerimientos

1. Definir las dimensiones estáticas de prueba:

```
#define M 3  
#define K 4
```

2. Declarar la matriz original A de dimensión MxK y la matriz transpuesta A^T de dimensión KxM:

```
float A[M][K];  
float A_T[K][M];
```

Nota: La matriz transpuesta invierte sus dimensiones a KxM.

3. Implementar la función de transposición:

```
void transponer_estatica(int m, int k, const float orig[][K], float dest[][M]);
```

Nota: analizar los argumentos. ¿Por qué el 'const'?

4. En la función `main`, inicializar A con valores de prueba, realizar la transposición e imprimir ambas matrices.

Ejercicio integrador: transponer matrices.

Requerimientos

1. Definir las dimensiones estáticas de prueba:

```
#define M 3  
#define K 4
```

2. Declarar la matriz original A de dimensión $M \times K$ y la matriz transpuesta

```
float A[M][K];  
float A_T[K][M];
```

Nota: La matriz transpuesta invierte sus dimensiones a $K \times M$.

3. Implementar la función de transposición:

```
void transponer_estatica(int m, int k, const float orig[][K], float d
```

Nota: analizar los argumentos. ¿Por qué el 'const'?

4. En la función `main`, inicializar A con valores de prueba, realizar la tran



Ejercicio integrador II: pasar a memoria dinámica.

Cuando M y K no se conocen hasta el tiempo de ejecución, se puede reservar un único bloque continuo de memoria en el Heap y calcular explícitamente las posiciones mediante aritmética de punteros sobre un arreglo 1D.

A =

1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

- $A[1]=2$ $A[8]=9$
- $A[1][2]=$ ¡¡está prohibido porque no se cuantas columnas tiene!!

Ejercicio integrador II: pasar a memoria dinámica.

Cuando M y K no se conocen hasta el tiempo de ejecución, se puede reservar un único bloque continuo de memoria en el Heap y calcular explícitamente las posiciones mediante aritmética de punteros sobre un arreglo 1D.

A =

fila 0				fila 1				fila 2			
1	2	3	4	5	6	7	8	9	10	11	12
col0	col1	col2	col3	col0	col1	col2	col3	col0	col1	col2	col3

- $A[1]=2$ y $A[9]=10$; pero $Ac[1][2]=?$ $Ac[2][0]=?$
- En general: $Ac[i][j] = A[...]$?

1	2	3	4
5	6	7	8
9	10	11	12

Ac es “el vector A interpretado como matriz cuadrada Ac”.

Ejercicio integrador II: pasar a memoria dinámica.

Requerimientos

1. Solicitar las dimensiones M y K al usuario.
2. Asignar memoria dinámica para la matriz original de MxK flotantes usando malloc.
3. Validar inmediatamente si la asignación fue exitosa:

```
if (orig == NULL) {  
    // Tratar error de memoria  
}
```

4. Asignar memoria dinámica para la matriz transpuesta de KxM flotantes y validar la asignación.
5. Cargar los datos de la matriz de entrada.
6. Implementar la función de transposición dinámica:

```
void transponer_dinamica(int m, int k, const float *orig, float *dest);
```

Ejercicio integrador II: pasar a memoria dinámica.

Requerimientos

1. Solicitar las dimensiones M y K al usuario.
2. Asignar memoria dinámica para la matriz original de MxK flotantes usando malloc.
3. Validar inmediatamente si la asignación fue exitosa:

```
if (orig == NULL) {  
    // Tratar error de memoria  
}
```

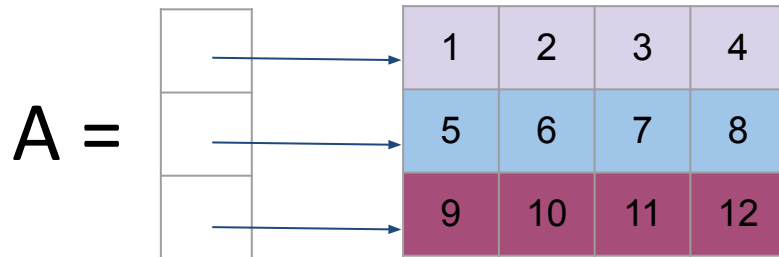
4. Asignar memoria dinámica para la matriz transpuesta de KxM flotantes.
5. Cargar los datos de la matriz de entrada.
6. Implementar la función de transposición dinámica:

```
void transponer_dinamica(int m, int k, const
```



Ejercicio integrador III: pasar a memoria dinámica II.

Otra forma de tomar memoria dinámica es usar un lazo y pedir M arreglos de K elementos para una matriz de MxK.



- $A[i]$ es un puntero.
- $A[i][j]$ es un valor

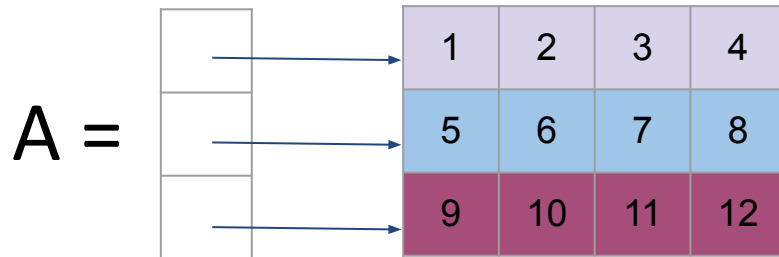
```
int M = 3; // Número de filas
int K = 4; // Número de columnas

// 1. Reservar el arreglo principal de M punteros
int **matriz = (int **)malloc(M * sizeof(int *));
if (matriz == NULL) {
    fprintf(stderr, "Error al asignar memoria para las filas.\n");
    return 1;
}

// 2. Reservar cada una de las M filas (de K elementos cada una)
for (int i = 0; i < M; i++) {
    matriz[i] = (int *)malloc(K * sizeof(int));
    if (matriz[i] == NULL) {
        fprintf(stderr, "Error al asignar memoria para la fila %d.\n", i);
        // Liberar lo asignado previamente en caso de fallo parcial
        for (int j = 0; j < i; j++) {
            free(matriz[j]);
        }
        free(matriz);
        return 1;
    }
}
```

Ejercicio integrador III: pasar a memoria dinámica II.

Otra forma de tomar memoria dinámica es usar un lazo y pedir M arreglos de K elementos para una matriz de MxK.



- $A[i]$ es un puntero.
- $A[i][j]$ es un valor

```
int M = 3; // Número de filas
int K = 4; // Número de columnas
```

```
// 1. Reservar el arreglo
int **matriz = (int **)
if (matriz == NULL) {
    fprintf(stderr, "Er
    return 1;
}
```

```
// 2. Reservar cada una
for (int i = 0; i < M;
    matriz[i] = (int *)
    if (matriz[i] == NULL)
        fprintf(stderr,
        // Liberar lo as
        for (int j = 0; j
            free(matriz[j]
        }
    free(matriz);
    return 1;
}
```



FIN